Kaunas University of Technology
Faculty of Informatics

Machine Learning Algorithms

Laboratory work 2

Student

IFDU-4 Denys Suntsov

Prof.

Čalnerytė Dalia

Vaitkutė Auksė

Kaunas, 2026

TABLE OF CONTENTS

1. Introduction 3

2. Dataset 3

3. Data Quality Analysis 3

3.1. Summary 3

3.2. Data quality issues 4

4. Regression decision tree 5

4.1. Hyperparameters 6

4.2. Hyperparameters testing 6

5. Classification decision tree 8

5.1. Hyperparameters testing 9

6. Regression random forest 13

6.1. Hyperparameters 13

6.2. Hyperparameters testing 13

7. Conclusion 16

1. Introduction

Main objective is to build, evaluate, and deploy regression and classification decision trees to predict housing prices ("Price") from the Melbourne Housing Market dataset.

2. Dataset

My dataset was Melbourne Housing Market with Price column that I need to predict.

3. Data Quality Analysis

3.1. Summary

My dataset consists of 8 categorical features ('Suburb', 'Address', 'Type', 'Method', 'SellerG', 'Date', 'Regionname', 'CouncilArea') and 5 numerical features ('Rooms', 'Price', 'Propertycount', 'Distance', 'Postcode').

Numerical features:

Numerical test df:

	column	N	% missing	cardinality	min	Q1	median	Q3	max	mean	std
0	Rooms	4844	0.0	11	1.0	2.0	3.0	4.0	31.0	3.054913	1.036540
1	Price	4844	0.0	1306	185000.0	607000.0	815000.0	1200000.0	6370000.0	984426.000619	589184.184293
2	Postcode	4844	0.0	193	3000.0	3051.0	3101.0	3163.0	3977.0	3122.665566	125.948416
3	Propertycount	4844	0.0	310	96.0	4242.0	6795.0	10412.0	21650.0	7604.094137	4450.148207
4	Distance	4844	0.0	157	0.0	7.0	11.4	17.2	49.3	12.743559	7.600511

Numerical validation df:

	column	N	% missing	cardinality	min	Q1	median	Q3	max	mean	std
0	Rooms	4843	0.0	9	1.0	2.0	3.0	4.0	10.0	3.052034	0.947277
1	Price	4843	0.0	1333	112000.0	608000.0	835000.0	1210000.0	11200000.0	994369.874045	617291.646016
2	Postcode	4843	0.0	201	3000.0	3052.0	3102.0	3163.0	3978.0	3124.093537	130.496050
3	Propertycount	4843	0.0	319	122.0	4280.0	6567.0	10412.0	21650.0	7546.320875	4454.973518
4	Distance	4843	0.0	163	0.0	7.0	11.7	16.5	47.4	12.689593	7.480172

Categorical features:

Categorical test df:

	column	N	% missing	cardinality	mode	mode freq	mode %	mode 2	mode 2 freq	mode 2 %
0	Suburb	4844	0.0	314	Reservoir	101	2.09	Craigieburn	65	1.34
1	Address	4844	0.0	4778	4/8 Chaleyer St	3	0.06	3/395 Glenferrie Rd	3	0.06
2	Type	4844	0.0	3	h	3391	70.00	u	975	20.13
3	Method	4844	0.0	5	S	3055	63.07	SP	671	13.85
4	SellerG	4844	0.0	221	Barry	460	9.50	Jellis	403	8.32
5	Date	4844	0.0	109	24/03/2018	87	1.80	18/03/2017	85	1.75
6	Regionname	4844	0.0	8	Northern Metropolitan	1385	28.59	Southern Metropolitan	1214	25.06
7	CouncilArea	4844	0.0	33	Boroondara City Council	353	7.29	Darebin City Council	350	7.23

Categorical validation df:

	column	N	% missing	cardinality	mode	mode freq	mode %	mode 2	mode 2 freq	mode 2 %
0	Suburb	4843	0.0	323	Reservoir	110	2.27	Bentleigh East	88	1.82
1	Address	4843	0.0	4777	90 MacEdon Rd	3	0.06	8/183 Shaws Rd	3	0.06
2	Type	4843	0.0	3	h	3417	70.56	u	926	19.12
3	Method	4843	0.0	5	S	3119	64.40	SP	649	13.40
4	SellerG	4843	0.0	207	Barry	489	10.10	Jellis	410	8.47
5	Date	4843	0.0	111	24/03/2018	97	2.00	14/10/2017	79	1.63
6	Regionname	4843	0.0	8	Northern Metropolitan	1380	28.49	Southern Metropolitan	1241	25.62
7	CouncilArea	4843	0.0	33	Darebin City Council	376	7.76	Boroondara City Council	308	6.36

After handling data quality issues, dataset will be split into train, validation and test datasets in ratio 80:10:10.

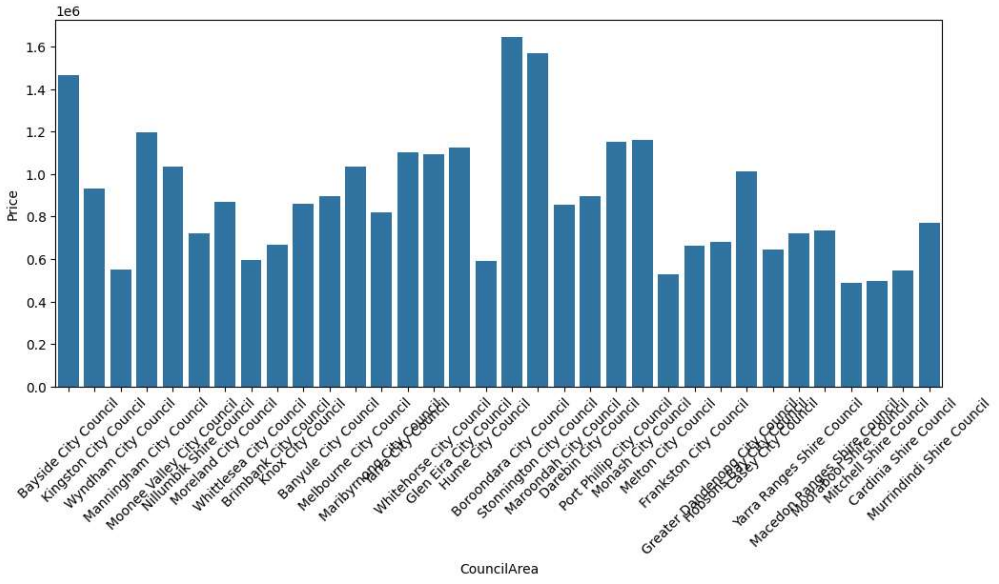
3.2. Data quality issues

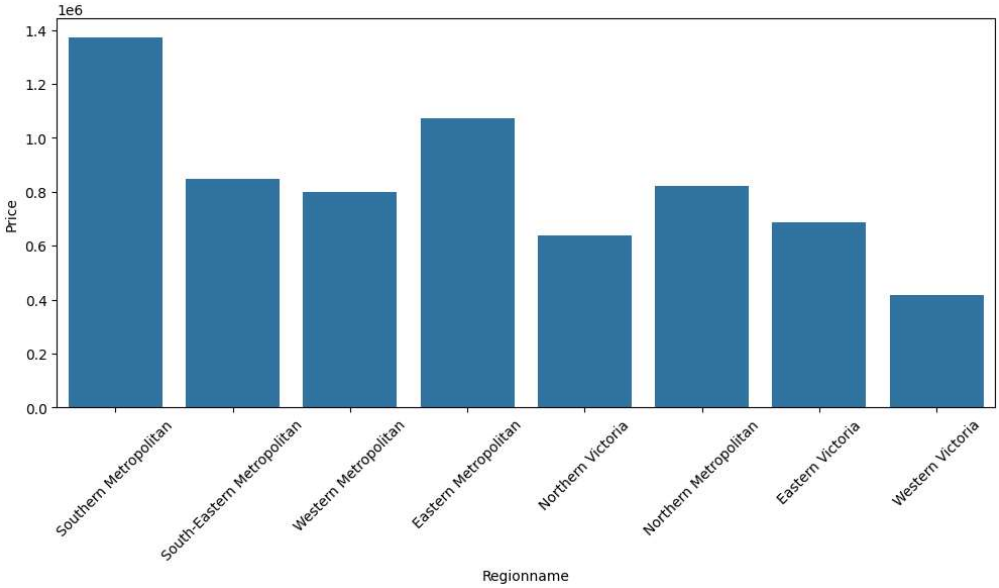
The column ‘Price’ that we want to predict has big percent of missing values, and we will drop the rows with them, because model can’t learn without a teacher.

Column ‘Rooms’ has a median of 3 rooms and an outlier of maximum of 31 rooms. I will retain such rows not to lose real data.

Columns ‘Suburb’, ‘SellerG’, ‘Address’, ‘Postcode’ show high cardinality, and I decided to drop those columns, because keeping them could lead to an excessive number of branches in the decision tree, resulting in more complex trees and increasing the risk of overfitting. ‘Address’ and ‘Postcode’ contain highly specific information about individual properties, and they are not producing any patterns for the model to learn from.

Columns “CouncilArea” and “Regionname” also have high cardinality, but they also have big influence on “Price” column.





I decided to keep those two categories without grouping, because different regions have distinct property price distributions. Grouping could lead to information loss and reduction of predictive accuracy.

Column 'Date' is formatted as raw string with 112 unique auction dates. We will extract 'Age' from it to capture temporal housing market trends as proper numerical features.

Also, 2 duplicate rows were found in the dataset, and they were removed to avoid repetiveness of data. I verified columns Price, Rooms, Distance, Propertycount for invalid values (e.g price, number of rooms etc. cannot be negative), and they were not found in dataset.

```
Price <= 0: 0
Rooms <= 0: 0
Distance < 0: 0
Propertycount < 0: 0
```

As for outliers in 'Price' variable, I decided to remove them to reduce the influence of extreme values on the tree's splitting criteria and minimize the risk of overfitting to noise.

```
Lower bound: -272500.0
Upper bound: 2107500.0
Number of outliers: 1832
```

4. Regression decision tree

A Regression Decision Tree is a non-parametric supervised learning algorithm used to predict continuous target variables. Regression Tree recursively partitions the feature space into non-overlapping regions $(R_1, R_2, \ldots, R_n)$ and assigns a continuous value to each region (tree's prediction $\hat{y}$), typically equal to the mean of the training target values within that region:

$$\hat{y} = \frac{1}{N_m} \sum_{i \in R_m} y_i$$

At each node, the decision tree searches across input features (j) and candidate split thresholds (s) for numerical variables to partition the data into two child nodes.

$$R_1(j,s) = \{X \mid X_j \leq s\} \quad \text{and} \quad R_2(j,s) = \{X \mid X_j > s\}$$

For categorical variables, the data is partitioned into multiple child nodes, one for each category.

The optimal feature j and threshold s are selected by minimizing the sum of squared errors across both child regions:

$$\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} \left(y_i - \hat{y}_{R_1}\right)^2 + \sum_{x_i \in R_2(j,s)} \left(y_i - \hat{y}_{R_2}\right)^2 \right]$$

While splitting numerical features, candidate split points (s) are evaluated at percentiles of the feature values, with at least five candidate threshold positions generated. For categorical variables, we create a separate branch for each category without converting categorical values into numerical values.

For binary variables, two branches are created. The optimal split is selected by minimizing the sum of squared errors (SSE) across all resulting child nodes.

4.1. Hyperparameters

Decision trees grown without constraints will expand until every leaf node contains a single sample, causing severe overfitting. To ensure generalization on unseen validation/test data, regularizing hyperparameters are applied:

- max_depth: limits the maximum vertical depth from root to leaf to constrain tree complexity.
- min_samples_split: sets the minimum number of samples required inside a node before a split is allowed.
- num_thresholds: restricts the number of candidate quantile thresholds evaluated per continuous numerical feature to speed up training.

4.2. Hyperparameters testing

During training, I chose 5 combinations for values of decision tree parameters:

max_depth	3	5	8	12	1000
min_samples_split	2	2	10	10	2
num_thresholds	5	5	10	10	20

Training produced such results:

max_depth	min_samples_split	num_thresholds	Train_MAE	Val_MAE	Train_MAPE	Val_MAPE	Train_R2	Val_R2
3	2	5	167467.731618	227032.526149	19.771439	21.078460	0.662117	0.500110
5	2	5	141164.019799	202453.839403	16.233290	18.290997	0.750145	0.562986
8	10	10	120918.364746	196147.236847	13.772440	17.502135	0.810772	0.569809
12	10	10	114855.985774	197961.503804	13.051348	17.739727	0.826336	0.568441
1000	2	20	96065.363303	201742.189336	10.808517	18.144330	0.858798	0.564779

According to error metrics, the best combination of parameters is 3, because validation MAE and MAPE stop improving at this point and are among lowest ones; R2 is the highest, which means that model has the best predictive performance.

Also, I ran those combinations of parameters on sklearn DesicionTreeRegressor:

max_depth	min_samples_split	Train_MAE	Val_MAE	Train_MAPE	Val_MAPE	Train_R2	Val_R2
3	2	231317.095936	294873.535679	28.576150	29.435643	0.417838	0.335916
5	2	188397.401672	246826.707306	22.756545	23.801050	0.598581	0.465040
8	10	157030.941515	213477.786291	18.657675	19.955525	0.708883	0.547815
12	10	132805.612334	200128.608051	15.553547	18.297242	0.783905	0.570255
1000	2	95570.369521	203100.030809	10.726346	18.380617	0.859785	0.564965

In case of sklearn implementation, the best tree depth is 12, as validation MAE stops improving at this point, and this depth shows the largest R2.

Sklearn implementation produced slightly larger errors for validation dataset in comparison to our custom implementation, which might be because it uses one-hot encoding for handling categorical features, as binary trees can evaluate one indicator variable per split, requiring deeper trees. Our custom

implementation splits categorical features into N child branches simultaneously, which shows better results at lower depths.

The best tree has such first 3 layers:

```
DECISION TREE: FIRST THREE LAYERS (depths 0, 1, 2)
ROOT: CouncilArea (categorical split) [n=36913, SSE=5799201658199986.00]
  CouncilArea = Bayside City Council: Type (categorical split) [n=1684, SSE=315851195143125.88]
    Type = h: Distance <= 17.30 [n=969, SSE=128098774228069.92]
    Type = t: Distance <= 17.30 [n=350, SSE=48774893583750.00]
    Type = u: Rooms <= 2.00 [n=365, SSE=24462581702945.20]
  CouncilArea = Kingston City Council: Type (categorical split) [n=1594, SSE=148693728618030.06]
    Type = u: Rooms <= 1.00 [n=324, SSE=7401681067766.36]
    Type = t: Rooms <= 2.00 [n=267, SSE=18421130521246.32]
    Type = h: Propertycount <= 4734.00 [n=1003, SSE=80503028688386.62]
  CouncilArea = Wyndham City Council: Rooms <= 3.00 [n=935, SSE=15321849104877.75]
    Rooms <= 3.00: Type (categorical split) [n=567, SSE=4742440076406.12]
    Rooms > 3.00: Age <= 8.00 [n=368, SSE=7064328219198.91]
  CouncilArea = Manningham City Council: Type (categorical split) [n=1371, SSE=129847806573793.81]
    Type = h: Rooms <= 3.00 [n=1081, SSE=84214580497426.75]
    Type = t: Rooms <= 3.00 [n=162, SSE=8529901356200.00]
    Type = u: Rooms <= 2.00 [n=128, SSE=3317621447880.47]
  CouncilArea = Moonee Valley City Council: Type (categorical split) [n=1702, SSE=281390430736321.34]
    Type = h: Distance <= 8.20 [n=1119, SSE=140768921012666.94]
    Type = t: Rooms <= 2.00 [n=196, SSE=8680374677295.92]
    Type = u: Rooms <= 2.00 [n=387, SSE=14926053744186.04]
  CouncilArea = Nillumbik Shire Council: Propertycount <= 1058.00 [n=195, SSE=13236654748717.95]
    Propertycount <= 1058.00: Rooms <= 4.00 [n=42, SSE=4740475529761.91]
    Propertycount > 1058.00: Rooms <= 4.00 [n=153, SSE=4553168264705.88]
  CouncilArea = Moreland City Council: Type (categorical split) [n=1968, SSE=226821019665401.00]
    Type = h: Distance <= 5.20 [n=1224, SSE=130210694228541.97]
    Type = t: Distance <= 5.20 [n=365, SSE=10568666309639.61]
    Type = u: Rooms <= 1.00 [n=379, SSE=7704996914854.88]
  CouncilArea = Whittlesea City Council: Type (categorical split) [n=1638, SSE=29425527186877.84]
    Type = h: Age <= 9.00 [n=1446, SSE=22413838501539.43]
    Type = t: Age <= 9.00 [n=55, SSE=272922734090.91]
    Type = u: Rooms <= 2.00 [n=137, SSE=1631722710510.95]
  CouncilArea = Brimbank City Council: Rooms <= 3.00 [n=2160, SSE=67520760848538.50]
    Rooms <= 3.00: Type (categorical split) [n=1456, SSE=33476313073521.68]
    Rooms > 3.00: Distance <= 11.70 [n=704, SSE=25412287379237.93]
  CouncilArea = Knox City Council: Rooms <= 3.00 [n=610, SSE=31552335973659.32]
    Rooms <= 3.00: Rooms <= 2.00 [n=307, SSE=8662242743713.15]
    Rooms > 3.00: Distance <= 14.70 [n=303, SSE=14864693308211.93]
  CouncilArea = Banyule City Council: Rooms <= 3.00 [n=2263, SSE=189775917702505.56]
    Rooms <= 3.00: Type (categorical split) [n=1583, SSE=94401649837064.62]
    Rooms > 3.00: Distance <= 8.90 [n=680, SSE=65938919707823.91]
  CouncilArea = Melbourne City Council: Rooms <= 2.00 [n=1573, SSE=292308892213248.31]
    Rooms <= 2.00: Type (categorical split) [n=1075, SSE=102915639391936.66]
    Rooms > 2.00: Type (categorical split) [n=498, SSE=67871340319437.75]
  CouncilArea = Maribyrnong City Council: Type (categorical split) [n=1413, SSE=144803278267831.44]
    Type = h: Rooms <= 3.00 [n=911, SSE=65719211531539.25]
    Type = t: Rooms <= 2.00 [n=222, SSE=6280018614594.59]
    Type = u: Rooms <= 1.00 [n=280, SSE=3910117378482.57]
  CouncilArea = Yarra City Council: Type (categorical split) [n=993, SSE=166851050820599.38]
    Type = u: Rooms <= 1.00 [n=354, SSE=27138897862577.68]
    Type = h: Rooms <= 2.00 [n=549, SSE=51264132359659.02]
    Type = t: Rooms <= 2.00 [n=90, SSE=5593433655555.56]
  CouncilArea = Whitehorse City Council: Type (categorical split) [n=1063, SSE=96361602008076.88]
    Type = h: Rooms <= 3.00 [n=834, SSE=61461233988865.88]
    Type = u: Rooms <= 2.00 [n=149, SSE=5015426741744.97]
    Type = t: Distance <= 14.70 [n=80, SSE=1621310955500.00]
  CouncilArea = Glen Eira City Council: Type (categorical split) [n=1828, SSE=331718493493009.12]
    Type = t: Rooms <= 3.00 [n=373, SSE=24005704885192.80]
    Type = h: Rooms <= 3.00 [n=917, SSE=93819044720731.94]
    Type = u: Rooms <= 2.00 [n=538, SSE=28660293922193.30]
  CouncilArea = Hume City Council: Rooms <= 3.00 [n=1888, SSE=59094736621213.11]
    Rooms <= 3.00: Distance <= 13.10 [n=1148, SSE=17879668769414.17]
    Rooms > 3.00: Propertycount <= 5070.00 [n=740, SSE=32550886602824.56]
```

```
CouncilArea = Boroondara City Council: Type (categorical split) [n=2094, SSE=563444071454524.38]
   Type = h: Rooms <= 2.00 [n=1068, SSE=105774581810187.28]
   Type = t: Rooms <= 2.00 [n=247, SSE=26634046168583.00]
   Type = u: Rooms <= 2.00 [n=779, SSE=54079371760041.05]
CouncilArea = Stonnington City Council: Type (categorical split) [n=732, SSE=192626795463126.56]
   Type = u: Rooms <= 2.00 [n=384, SSE=50308443683406.15]
   Type = t: Propertycount <= 7717.00 [n=59, SSE=5955965720338.98]
   Type = h: Rooms <= 2.00 [n=289, SSE=32531824837370.24]
CouncilArea = Maroondah City Council: Rooms <= 3.00 [n=812, SSE=42002743806530.80]
   Rooms <= 3.00: Type (categorical split) [n=516, SSE=17345693273229.50]
   Rooms > 3.00: Distance <= 21.30 [n=296, SSE=16219688987588.91]
CouncilArea = Darebin City Council: Type (categorical split) [n=2678, SSE=316617318395137.56]
   Type = u: Rooms <= 1.00 [n=496, SSE=8774973439301.09]
   Type = h: Distance <= 5.70 [n=1805, SSE=188077403437256.44]
   Type = t: Propertycount <= 14577.00 [n=377, SSE=10041375798267.90]
CouncilArea = Port Phillip City Council: Type (categorical split) [n=927, SSE=235381902803439.16]
   Type = u: Rooms <= 1.00 [n=565, SSE=27869376854481.00]
   Type = h: Rooms <= 2.00 [n=322, SSE=42638731656055.91]
   Type = t: Rooms <= 2.00 [n=40, SSE=3190649493750.00]
CouncilArea = Monash City Council: Type (categorical split) [n=1910, SSE=215271178210953.88]
   Type = u: Rooms <= 2.00 [n=261, SSE=11566330243797.09]
   Type = h: Distance <= 16.70 [n=1392, SSE=1379797773837 99.94]
   Type = t: Rooms <= 3.00 [n=257, SSE=12600541329988.33]
CouncilArea = Melton City Council: Regionname (categorical split) [n=439, SSE=12850361747874.03]
   Regionname = Western Metropolitan: Rooms <= 3.00 [n=230, SSE=5049203330057.00]
   Regionname = Western Victoria: Rooms <= 4.00 [n=139, SSE=2084745756294.96]
   Regionname = Northern Victoria: Age <= 9.00 [n=70, SSE=496268585714.29]
CouncilArea = Frankston City Council: Rooms <= 3.00 [n=531, SSE=28020513617909.84]
   Rooms <= 3.00: Type (categorical split) [n=369, SSE=13574929872791.56]
   Rooms > 3.00: Distance <= 34.95 [n=162, SSE=11291301156443.09]
CouncilArea = Greater Dandenong City Council: Rooms <= 2.00 [n=465, SSE=15371786760241.00]
   Rooms <= 2.00: Distance <= 20.80 [n=67, SSE=1249234382985.07]
   Rooms > 2.00: Rooms <= 3.00 [n=398, SSE=10534177506500.50]
CouncilArea = Hobsons Bay City Council: Type (categorical split) [n=873, SSE=105178348988608.25]
   Type = u: Rooms <= 3.00 [n=109, SSE=3724065862385.32]
   Type = h: Distance <= 6.80 [n=652, SSE=72484213337465.48]
   Type = t: Rooms <= 3.00 [n=112, SSE=5052761271071.43]
CouncilArea = Casey City Council: Rooms <= 3.00 [n=264, SSE=9890156576678.53]
   Rooms <= 3.00: Propertycount <= 8256.00 [n=154, SSE=3142310766639.61]
   Rooms > 3.00: Propertycount <= 9376.00 [n=110, SSE=4603900193997.96]
CouncilArea = Yarra Ranges Shire Council: Rooms <= 3.00 [n=142, SSE=4501445543338.46]
   Rooms <= 3.00: Distance <= 27.10 [n=80, SSE=1153489535500.00]
   Rooms > 3.00: Distance <= 30.88 [n=62, SSE=2539131736280.98]
CouncilArea = Macedon Ranges Shire Council: Rooms <= 3.00 [n=94, SSE=5723875117021.28]
   Rooms <= 3.00: Method (categorical split) [n=51, SSE=1807905656862.75]
   Rooms > 3.00: Propertycount <= 401.80 [n=43, SSE=2706126918604.65]
CouncilArea = Moorabool Shire Council: Rooms <= 3.00 [n=10, SSE=217358100000.00]
   Rooms <= 3.00: LEAF: prediction=446888.8888888889 [n=9, SSE=52406888888.89]
   Rooms > 3.00: LEAF: prediction=875000.0 [n=1, SSE=0.00]
CouncilArea = Mitchell Shire Council: Rooms <= 3.00 [n=21, SSE=374912571428.57]
   Rooms <= 3.00: Age <= 9.00 [n=10, SSE=27144000000.00]
   Rooms > 3.00: Rooms <= 4.00 [n=11, SSE=149702727272.73]
CouncilArea = Cardinia Shire Council: Rooms <= 4.00 [n=42, SSE=858779642857.14]
   Rooms <= 4.00: Propertycount <= 2768.00 [n=38, SSE=603403052631.58]
   Rooms > 4.00: LEAF: prediction=732750.0 [n=4, SSE=103890750000.00]
CouncilArea = Murrindindi Shire Council: LEAF: prediction=770000.0 [n=1, SSE=0.00]
```

5. Classification decision tree

A Classification Decision Tree is a non-parametric supervised learning algorithm used to predict categorical target variables by partitioning the feature space into distinct, non-overlapping region nodes. Each leaf node in a classification tree assigns a discrete class label $\hat{y}$ based on the majority class (plurality vote) or outputs a discrete probability distribution over all target classes:

$$\hat{y} = \arg\max_{k} \hat{p}_{m,k}$$

Where $\hat{p}_{m,k}$ represents the proportion of training observations in node m belonging to class k:

$$\hat{p}_{m,k} = \frac{1}{N_m} \sum_{x_i \in R_m} I\,(y_i = k)$$

At each node m, the decision tree evaluates candidate splits across continuous and categorical features to maximize the reduction in node impurity. In my case, I use Gini impurity:

$$G(m) = 1 - \sum_{k=1}^{K} \left(\hat{p}_{m,k}\right)^2$$

Tree expansion stops immediately if node impurity reaches zero, indicating a completely pure node.

The error in such tree is evaluated by using:

- Accuracy – proportion of total predictions that the model got correct:
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$
- Precision - out of all instances the model predicted as positive, how many were actually positive?
$$\text{Precision} = \frac{TP}{TP + FP}$$
- Sensitivity - out of all instances that were actually positive, how many did the model correctly identify?
$$\text{Sensitivity} = \frac{TP}{TP + FN}$$
- Specificity – out of all instances that were actually negative, how many did the model correctly identify?
$$\text{Specificity} = \frac{TN}{TN + FP}$$

- F1-score - shows balance between precision and sensitivity:
$$F_1 = 2 \times \frac{\text{Precision} \times \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}$$

To predict “Price” column, we have to mark continuous column to categorical values: e.g., price is very small (0), small (1), medium (2), large (3), very large (4).

5.1. Hyperparameters testing

I used exact same parameters combination as for regression decision tree, and they produced such results:

max_depth	min_samples_split	num_thresholds	Train_Acc	Val_Acc	Train_Precision	Val_Precision	Train_Sensitivity	Val_Sensitivity	Train_Specificity	Val_Specificity	Train_F1	Val_F1
3	2	5	0.533335	0.485236	0.540635	0.409289	0.534749	0.424272	0.883284	0.894001	0.534726	0.412940
5	2	5	0.601902	0.538303	0.603604	0.448862	0.602975	0.470168	0.900401	0.904923	0.602753	0.457641
8	10	10	0.668057	0.545736	0.672807	0.457789	0.668599	0.476995	0.916927	0.906453	0.670206	0.466293
12	10	10	0.690516	0.545943	0.696687	0.460002	0.690896	0.477290	0.922538	0.906502	0.692968	0.467263
1000	2	20	0.736461	0.542639	0.743738	0.459600	0.736587	0.474263	0.934024	0.905820	0.738868	0.465179

The best combination of parameters turned out to be 3, which achieved the best overall validation performance, with accuracy = 0.5457, precision = 0.4578, sensitivity = 0.4770, specificity = 0.9065, and

F1 = 0.4663. However, generalization to unseen data is not particularly strong, as there is a noticeable gap between training and validation accuracy (0.1223). At the same time, deeper trees achieve higher training accuracy (73.65%), but their validation F1 scores do not improve, which shows overfitting to the training data.

Training confusion matrix

	Pred_very small	Pred_small	Pred_medium	Pred_large	Pred_very large
Actual_very small	5646	1346	302	83	18
Actual_small	917	4818	1270	343	35
Actual_medium	199	1107	4394	1472	204
Actual_large	43	246	1000	5200	1102
Actual_very large	11	86	251	1389	5431

Validation confusion matrix

	Pred_very small	Pred_small	Pred_medium	Pred_large	Pred_very large
Actual_very small	618	213	61	22	2
Actual_small	148	497	220	65	4
Actual_medium	35	186	392	222	29
Actual_large	11	51	174	511	213
Actual_very large	2	13	46	246	626

Confusion matrix also shows good outputs, because highest numbers in both of them are located in the main diagonal, and misclassifications are entirely confined to adjacent classes, there are almost no extreme misclassifications.

The tree was also trained using sklearn library, and produced different outputs:

max_depth	min_samples_split	num_thresholds	Train_Acc	Val_Acc	Train_Precision	Val_Precision	Train_Sensitivity	Val_Sensitivity	Train_Specificity	Val_Specificity	Train_F1	Test_F1	Val_F1
3	2	5	0.377455	0.362792	0.440046	0.328050	0.377245	0.313826	0.844172	0.868420	0.363767	0.287751	0.292818
5	2	5	0.461789	0.439397	0.506934	0.384743	0.461550	0.380291	0.865233	0.884213	0.451603	0.363555	0.358882
8	10	10	0.543657	0.507537	0.542418	0.416181	0.544218	0.443590	0.885796	0.898629	0.541139	0.417884	0.428248
12	10	10	0.623927	0.545943	0.632035	0.458028	0.624582	0.476797	0.905894	0.906483	0.626562	0.463039	0.465115
1000	2	20	0.738168	0.545323	0.745208	0.460822	0.738295	0.476555	0.934452	0.906380	0.740504	0.465304	0.467087

In this case, the best tree is 4, because we can see improvement in Test_F1 (0.463566) and Val_F1 (0.464898) in comparison to previous tree configurations. With deeper tree (max_depth=1000), accuracy change is low, so there is no reason for having bigger trees.

The reason for difference between custom and sklearn classification trees is the same as for regression trees: custom implementation handles categorical features directly, while sklearn applies one-hot encoding.

The best tree has such first tree layers:

```
DECISION TREE: FIRST THREE LAYERS (depths 0, 1, 2)
ROOT: CouncilArea (categorical split) [n=36913, gini=0.7999]
  CouncilArea = Bayside City Council: Type (categorical split) [n=1684, gini=0.6905]
    Type = h: Distance <= 17.30 [n=969, gini=0.5127]
    Type = t: Distance <= 17.30 [n=350, gini=0.6881]
    Type = u: Rooms <= 2.00 [n=365, gini=0.7559]
  CouncilArea = Kingston City Council: Type (categorical split) [n=1594, gini=0.7635]
    Type = u: Rooms <= 1.00 [n=324, gini=0.6392]
    Type = t: Rooms <= 2.00 [n=267, gini=0.7331]
    Type = h: Propertycount <= 4734.00 [n=1003, gini=0.6985]
  CouncilArea = Wyndham City Council: Rooms <= 3.00 [n=935, gini=0.5126]
    Rooms <= 3.00: Propertycount <= 10160.00 [n=567, gini=0.3358]
    Rooms > 3.00: Distance <= 14.70 [n=368, gini=0.6230]
  CouncilArea = Manningham City Council: Type (categorical split) [n=1371, gini=0.6677]
    Type = h: Propertycount <= 5420.00 [n=1081, gini=0.5977]
```

Type = t: Rooms <= 3.00 [n=162, gini=0.6686]
Type = u: Rooms <= 2.00 [n=128, gini=0.6644]
CouncilArea = Moonee Valley City Council: Type (categorical split) [n=1702, gini=0.7808]
Type = h: Distance <= 8.20 [n=1119, gini=0.6850]
Type = t: Rooms <= 2.00 [n=196, gini=0.7064]
Type = u: Rooms <= 2.00 [n=387, gini=0.5809]
CouncilArea = Nillumbik Shire Council: Propertycount <= 4258.00 [n=195, gini=0.7407]
Propertycount <= 4258.00: Rooms <= 3.00 [n=107, gini=0.7120]
Propertycount > 4258.00: Rooms <= 3.00 [n=88, gini=0.5560]
CouncilArea = Moreland City Council: Type (categorical split) [n=1968, gini=0.7953]
Type = h: Distance <= 5.20 [n=1224, gini=0.7588]
Type = t: Distance <= 5.20 [n=365, gini=0.7066]
Type = u: Rooms <= 1.00 [n=379, gini=0.4893]
CouncilArea = Whittlesea City Council: Type (categorical split) [n=1638, gini=0.6118]
Type = h: Age <= 9.00 [n=1446, gini=0.6110]
Type = t: Distance <= 17.90 [n=55, gini=0.1031]
Type = u: Rooms <= 3.00 [n=137, gini=0.2206]
CouncilArea = Brimbank City Council: Type (categorical split) [n=2160, gini=0.6899]
Type = h: Age <= 9.00 [n=1852, gini=0.6813]
Type = t: Distance <= 12.90 [n=135, gini=0.6397]
Type = u: Rooms <= 2.00 [n=173, gini=0.3092]
CouncilArea = Knox City Council: Rooms <= 3.00 [n=610, gini=0.7066]
Rooms <= 3.00: Rooms <= 2.00 [n=307, gini=0.6569]
Rooms > 3.00: Distance <= 23.50 [n=303, gini=0.6246]
CouncilArea = Banyule City Council: Rooms <= 2.00 [n=2263, gini=0.7518]
Rooms <= 2.00: Type (categorical split) [n=407, gini=0.6796]
Rooms > 2.00: Rooms <= 3.00 [n=1856, gini=0.7222]
CouncilArea = Melbourne City Council: Type (categorical split) [n=1573, gini=0.7907]
Type = u: Rooms <= 1.00 [n=925, gini=0.7137]
Type = h: Rooms <= 2.00 [n=515, gini=0.6066]
Type = t: Regionname (categorical split) [n=133, gini=0.7459]
CouncilArea = Maribyrnong City Council: Type (categorical split) [n=1413, gini=0.7764]
Type = h: Rooms <= 3.00 [n=911, gini=0.7024]
Type = t: Rooms <= 2.00 [n=222, gini=0.6816]
Type = u: Rooms <= 2.00 [n=280, gini=0.1882]
CouncilArea = Yarra City Council: Type (categorical split) [n=993, gini=0.7624]
Type = u: Rooms <= 1.00 [n=354, gini=0.6895]
Type = h: Rooms <= 2.00 [n=549, gini=0.5894]
Type = t: Rooms <= 2.00 [n=90, gini=0.6630]
CouncilArea = Whitehorse City Council: Type (categorical split) [n=1063, gini=0.6966]
Type = h: Rooms <= 3.00 [n=834, gini=0.6185]
Type = u: Rooms <= 2.00 [n=149, gini=0.6926]
Type = t: Rooms <= 3.00 [n=80, gini=0.6147]
CouncilArea = Glen Eira City Council: Type (categorical split) [n=1828, gini=0.7456]
Type = t: Rooms <= 3.00 [n=373, gini=0.6567]
Type = h: Rooms <= 3.00 [n=917, gini=0.5481]
Type = u: Rooms <= 1.00 [n=538, gini=0.6732]
CouncilArea = Hume City Council: Rooms <= 3.00 [n=1888, gini=0.6086]
Rooms <= 3.00: Distance <= 13.10 [n=1148, gini=0.4924]
Rooms > 3.00: Propertycount <= 5070.00 [n=740, gini=0.6725]
CouncilArea = Boroondara City Council: Type (categorical split) [n=2094, gini=0.6645]
Type = h: Rooms <= 2.00 [n=1068, gini=0.2156]
Type = t: Rooms <= 2.00 [n=247, gini=0.6083]
Type = u: Rooms <= 1.00 [n=779, gini=0.7468]
CouncilArea = Stonnington City Council: Type (categorical split) [n=732, gini=0.7522]
Type = u: Rooms <= 1.00 [n=384, gini=0.7338]

```
    Type = t: Rooms <= 2.00 [n=59, gini=0.5901]
    Type = h: Rooms <= 2.00 [n=289, gini=0.3714]
CouncilArea = Maroondah City Council: Type (categorical split) [n=812, gini=0.7340]
    Type = h: Rooms <= 3.00 [n=678, gini=0.7029]
    Type = u: Rooms <= 2.00 [n=91, gini=0.5842]
    Type = t: Distance <= 19.90 [n=43, gini=0.6436]
CouncilArea = Darebin City Council: Type (categorical split) [n=2678, gini=0.7907]
    Type = u: Rooms <= 1.00 [n=496, gini=0.4521]
    Type = h: Distance <= 5.70 [n=1805, gini=0.7369]
    Type = t: Propertycount <= 14577.00 [n=377, gini=0.6976]
CouncilArea = Port Phillip City Council: Type (categorical split) [n=927, gini=0.7706]
    Type = u: Rooms <= 1.00 [n=565, gini=0.6457]
    Type = h: Rooms <= 2.00 [n=322, gini=0.4507]
    Type = t: Method (categorical split) [n=40, gini=0.6262]
CouncilArea = Monash City Council: Type (categorical split) [n=1910, gini=0.7109]
    Type = u: Rooms <= 2.00 [n=261, gini=0.7121]
    Type = h: Distance <= 16.70 [n=1392, gini=0.6402]
    Type = t: Rooms <= 3.00 [n=257, gini=0.6912]
CouncilArea = Melton City Council: Regionname (categorical split) [n=439, gini=0.5220]
    Regionname = Western Metropolitan: Rooms <= 3.00 [n=230, gini=0.6540]
    Regionname = Western Victoria: Rooms <= 4.00 [n=139, gini=0.0971]
    Regionname = Northern Victoria: Rooms <= 3.00 [n=70, gini=0.0555]
CouncilArea = Frankston City Council: Type (categorical split) [n=531, gini=0.6936]
    Type = h: Distance <= 34.90 [n=459, gini=0.7131]
    Type = u: Rooms <= 2.00 [n=48, gini=0.1866]
    Type = t: Method (categorical split) [n=24, gini=0.5799]
CouncilArea = Greater Dandenong City Council: Rooms <= 2.00 [n=465, gini=0.7112]
    Rooms <= 2.00: Distance <= 20.80 [n=67, gini=0.2669]
    Rooms > 2.00: Propertycount <= 8459.00 [n=398, gini=0.6887]
CouncilArea = Hobsons Bay City Council: Type (categorical split) [n=873, gini=0.7767]
    Type = u: Rooms <= 2.00 [n=109, gini=0.5434]
    Type = h: Distance <= 11.00 [n=652, gini=0.7410]
    Type = t: Rooms <= 2.00 [n=112, gini=0.6781]
CouncilArea = Casey City Council: Rooms <= 3.00 [n=264, gini=0.6817]
    Rooms <= 3.00: Age <= 9.00 [n=154, gini=0.5420]
    Rooms > 3.00: Propertycount <= 9376.00 [n=110, gini=0.7321]
CouncilArea = Yarra Ranges Shire Council: Rooms <= 3.00 [n=142, gini=0.6869]
    Rooms <= 3.00: Distance <= 27.10 [n=80, gini=0.6194]
    Rooms > 3.00: Age <= 9.00 [n=62, gini=0.6925]
CouncilArea = Macedon Ranges Shire Council: Method (categorical split) [n=94, gini=0.7508]
    Method = S: Rooms <= 3.00 [n=46, gini=0.7146]
    Method = SP: Rooms <= 3.00 [n=20, gini=0.6600]
    Method = VB: LEAF: prediction=small [n=8, gini=0.4688]
    Method = PI: Rooms <= 3.00 [n=18, gini=0.5802]
    Method = SA: LEAF: prediction=very large [n=2, gini=0.5000]
CouncilArea = Moorabool Shire Council: Rooms <= 3.00 [n=10, gini=0.1800]
    Rooms <= 3.00: LEAF: prediction=very small [n=9, gini=0.0000]
    Rooms > 3.00: LEAF: prediction=medium [n=1, gini=0.0000]
CouncilArea = Mitchell Shire Council: Rooms <= 3.00 [n=21, gini=0.4717]
    Rooms <= 3.00: LEAF: prediction=very small [n=10, gini=0.0000]
    Rooms > 3.00: Method (categorical split) [n=11, gini=0.5620]
CouncilArea = Cardinia Shire Council: Propertycount <= 2768.00 [n=42, gini=0.5204]
    Propertycount <= 2768.00: Rooms <= 4.00 [n=17, gini=0.6436]
    Propertycount > 2768.00: Age <= 8.00 [n=25, gini=0.3392]
CouncilArea = Murrindindi Shire Council: LEAF: prediction=medium [n=1, gini=0.0000]
```

6. Regression random forest

A single decision tree suffers from overfitting. Random Forest overcomes this by combining an ensemble of N decision trees. Each base tree is trained on a distinct bootstrap dataset generated by sampling $N_{samples}$ rows with replacement from the training set (X, y). Training individual trees on different subsets of data reduces the overall variance of the final model without increasing bias.

Standard decision trees evaluate all available features at every split. Random Forests introduce additional diversity by evaluating only a random subset of features (max_features="sqrt" or $\lfloor\sqrt{p}\rfloor$) at each node split. Without feature subsampling, a single dominant feature (e.g: distance) would be selected as the root node across almost all bootstrap trees, so all of the trees will be highly correlated. It‘s necessary to find alternative predictive relationships instead of using the same features accross all trees.

For regression tasks, final ensemble predictions are generated by computing the arithmetic mean of predictions across all B base trees:

$$\hat{y}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b\,(x)$$

where $T_b(x)$ represents the continuous prediction of the b-th regression tree.

6.1. Hyperparameters

I used such parameters while constructing random forest:

- n_trees: defines the total number of individual CustomRegressionTree instances constructed in the ensemble. As N_{trees} increases, the ensemble variance decreases and predictions stabilize.
- depth: controls the maximum vertical depth (max_depth) of each decision tree, regulating individual model complexity and preventing deep overfitting.
- min_split: sets the minimum number of samples (min_samples_split) required to attempt a split at any internal node within a base tree.

6.2. Hyperparameters testing

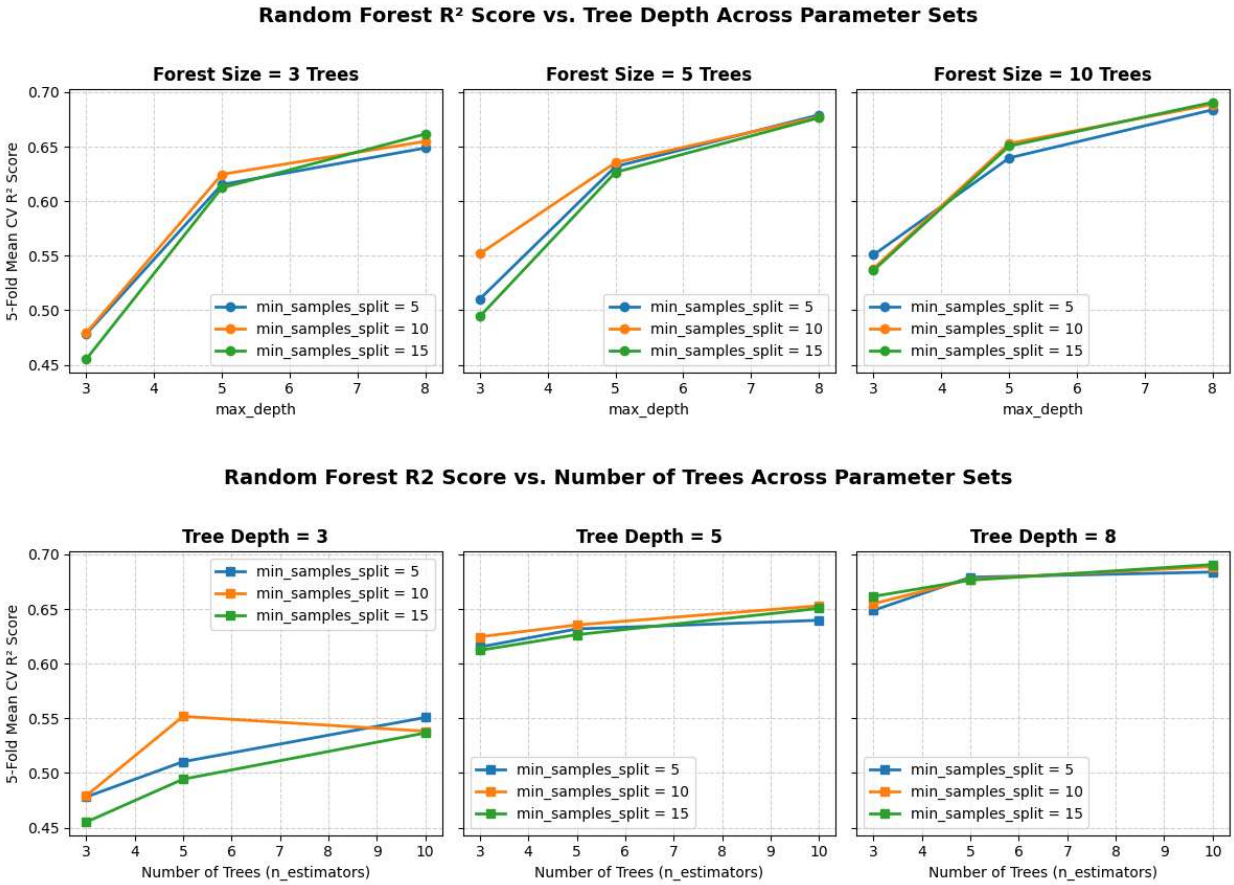
During training, I chose 3 combinations for values of random forest parameters:

n_trees	3	5	10
Depth	3	5	8
min_split	5	10	15

Overall, we have 27 configurations, for each of them 5-Fold cross validation was performed

After training, I printed out top 5 configurations (sorted according to MAE):

```
Top custom conf
    n_estimators  max_depth  min_samples_split    Mean_CV_MAE   Std_CV_MAE  Mean_CV_R2  Std_CV_R2
26            10          8                 15  157388.410213  1940.089943    0.690705   0.014385
25            10          8                 10  157521.875723  1674.758826    0.688802   0.011029
24            10          8                  5  159152.173513  2011.821702    0.683833   0.017080
15             5          8                  5  160573.828586  2205.398513    0.679115   0.011337
16             5          8                 10  161110.882051  1977.522469    0.677325   0.016422
```



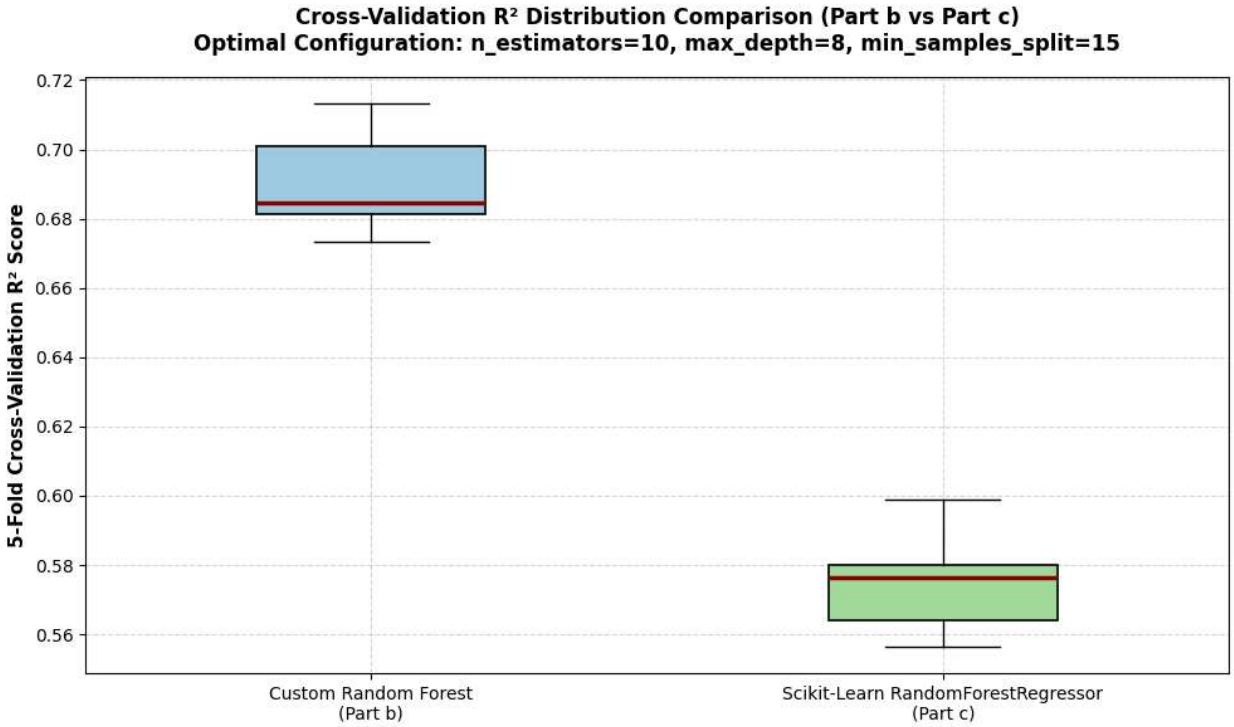
The best configuration (according to MAE and R2 criteria) is n_estimators = 10, max_depth = 8, min_samples_split = 15. As number of trees increases, predictive performance (R2) also increases, because each tree is trained on a different sample of data with random feature subsampling. If we average predictions across an increasing number of different trees, it will reduce model variance and individual prediction errors without increasing model error.

Testing those parameters on sklearn library, I got slightly larger errors:

```
Top 5 sklearn random forests:
n_estimators  max_depth  min_samples_split   Mean_CV_MAE  Std_CV_MAE  Mean_CV_R2  Std_CV_R2
          10          8                  5 193943.176338 2511.912210    0.577661   0.016368
          10          8                 15 195165.209475 2090.165382    0.575202   0.014535
          10          8                 10 195297.767528 1969.490893    0.573380   0.011304
           5          8                  5 197693.566090 5247.473453    0.565049   0.022085
           5          8                 15 198824.264326 2160.077568    0.563168   0.011477
```

If we compare R2 results for both custom and sklearn implementations on best configuration, we can observe that custom model has higher median predictive accuracy (median R2 = 0.69) compared to

sklearn RandomForestRegressor (median R2 = 0.57). Both models exhibit similar, tight box sizes, indicating consistent performance across folds. The difference in preformances between those models is likely the same as discussed above – the way of handling categorical variables is different.



Using those best parameters, I trained custom random forest and applied it for prediction on test dataset. The model produced such results:

```
Training model
Trained random forest with 10 trees, max_depth: 8, min_samples_split: 15
Evaluation metrics:
 Mean Absolute Error (MAE): 200,215.69
 Root Mean Squared Error (RMSE): 376,355.89
 Mean Absolute Percentage Error: 18.63%
 Coefficient of Determination (R2): 0.6054
Prediction:
           Suburb Type  Rooms Actual Price ($) Predicted Price ($) Absolute Error ($) Percentage Error (%)
0       Thornbury    h      2       $1,164,000           $949,588          $214,412               18.42%
1       Kurunjang    h      3         $381,000           $411,038           $30,038                7.88%
2         Preston    u      2         $530,000           $542,833           $12,833                2.42%
3         Newport    u      3         $850,000           $723,593          $126,407               14.87%
4        Oak Park    h      3         $892,000           $851,610           $40,390                4.53%
5  Mount Waverley    h      2       $1,240,000         $1,257,043           $17,043                1.37%
6         Glenroy    h      3         $450,000           $788,646          $338,646               75.25%
7         Niddrie    h      3       $1,015,000           $917,766           $97,234                9.58%
8         Croydon    h      5         $760,000           $828,287           $68,287                8.99%
9  Mount Waverley    h      2         $765,000         $1,270,183          $505,183               66.04%
```

We can see that MAPE across test data samples is 18.63%, and R2 is 0.6054, which shows good results. Some properties (e.g: Mount Waverley) show very close predictions (error ≈ 1.37%) to actual market sales. There are also properties which show higher error (e.g: Glenroy, error ≈ 75.25%), which might be because of unobserved real-estate variables not present in the dataset (e.g: renovation status, land condition).

7. Conclusion

In this laboratory work, custom decision tree models with native multi-way categorical splitting and ensemble random forests were built from scratch to predict housing prices using the Melbourne Housing Market dataset. I started by cleaning up the data - dropping missing target rows, trimming outliers, and collapsing high-cardinality location features so the models had a clear signal to learn from.

Testing different depth parameters showed how quickly decision trees can memorize training noise, highlighting why depth limits are essential for good generalization on unseen data.

Running 5-fold cross-validation on the random forest proved that averaging predictions across bootstrapped trees with random feature subsets effectively controls model variance. When benchmarked against scikit-learn, custom trees performed exceptionally well, showing that splitting categorical features into direct multi-way branches holds a strong structural edge over traditional one-hot encoding.

Lastly, I put together a standalone prediction script to load our trained forest model and predict prices for raw, unseen test samples.